dicators. These show a clear secondary peak near zero across models that contain the under-trained tokens, as well as high correlation between alternative indicators, further validating their effectiveness.

3 Results

In this section, we present a summary of our key findings on under-trained token detection. Table 1 presents verification statistics and examples of verified under-trained tokens for a wide range of models. The number of verified tokens varies significantly across different model families and tokenizer vocabulary sizes, and also depends on the number of unused special tokens that a model's tokenizer allows as plain-text input. The percentage of verified tokens typically ranges between 5–50% of tested candidate tokens, corresponding to 0.1–1% of the total vocabulary size.

Given the model-specific nature and the extensive volume of results, we elaborate on some common findings and showcase representative examples for particular models. Comprehensive reports covering an increasing number of tested models and token types are available in our repository.

3.1 Common observations

Although many of our findings are dependent on model-specific details such as tokenizer training and configuration, model architecture, and training data, there are a number of commonalities that appear across many different models.

3.1.1 Single-byte tokens

Tokens representing a single byte are a common source of untrained tokens. The most common occurrences are the bytes `0xF5`–`0xFF` which are not used in UTF-8 encoded text[2], and are a convenient source for quickly locating reference untrained tokens for indicators that require them. In addition, many tokenizers including those from the Gemma, Llama2 and Mistral families include every byte as a token, with many of them in the normal ASCII range `0x00`–`0x7F` being redundant and unreachable due to the existence of a token for the corresponding character.

These issues are not universal, and we also find models which include precisely the 243 bytes used in UTF-8 as tokens. Untrained single byte tokens are typically classified as 'partial UTF-8 sequences' or 'unreachable', and our indicators are effective in revealing which ones are never or rarely seen in model training.

3.1.2 Intermediate BPE fragments

All tested models use BPE-based tokenization, which retains the original tokens after a merge, often causing intermediate 'junk' tokens (Bostrom and Durrett, 2020). When mentioning examples of such as under-trained fragments, we denote the more complete token in parentheses, e.g. `_TheNitrome` (`_TheNitromeFan`) in the GPT-2 tokenizer. In some instances, the longest token is *also* under-trained, along with a variety of fragments. The same mechanism appears to explain many under-trained partial UTF-8 sequences in byte-level BPE tokenizers, with multiple bytes being merged over several steps, potentially leaving multiple intermediate tokens with partial Unicode characters.

3.1.3 Special tokens

Many models include untrained special tokens, such as `<pad>`, `<unk>`, or `<|unused_123|>`. In the following discussion we generally omit mentioning them, unless their status as an (un)trained token is particularly surprising, as their inclusion in the tokenizer and training data is typically deliberate, for purposes such as the ability to fine-tune models without changing tokenizers. One common observation is that, on many occasions, tokens such as `<mask>`, which we expect to be completely untrained, nevertheless appear to have been seen in training. A likely cause is code repositories or guides about language models using these tokens in normal text, along with tokenizers allowing such special control tokens in input text.

Special tokens can be unreachable due to input sanitization as well as configuration errors. In particular, both the Gemma and Yi models include special tokens relating to HTML tags, which were initially detected as unreachable, with the tags being split up in pre-tokenization.[3]

3.2 Model-specific observations

In this section we highlight notable model-specific observations, grouped by the tokenizer used. These examples are mainly intended to illustrate the variety of different under-trained tokens and

[2] See Appendix C for a primer on UTF-8 encoding.

[3] The Gemma team released a fix in response to our report, and the 01.AI team advise not to use the 'fast' version. Our reported results are based on the latest recommended versions.

Model	#Tokens	Tied Emb.	#Confirmed	Examples
GPT-2 Medium (0.4B)	50,257	Yes	49/999	`InstoreAndOnline` `reportprint` `_externalToEVA`
GPT-2 XL (1.5B)	50,257	Yes	67/999	`InstoreAndOnline` `_RandomRedditor` `embedreportprint`
GPT-J 6B	50,400	No*	200/999	`_attRot` `_externalToEVA` `_SolidGoldMagikarp`
Phi-2 (2.7B)	50,295	No*	103/999	`DragonMagazine` `_TheNitrome` `_SolidGoldMagikarp`
Pythia 6.7B	50,277	No	14/993	`FFIRMED` `_taxp` `_affidav`
GPT-NeoX 20B	50,277	No	10/993	`FFIRMED` `_taxp` `_affidav`
OLMo v1.7 7B	50,280	No	178/993	`_§\[` `medscimonit` `FFIRMED` `_[****`
Llama2 7B	32,000	No	20/639	`_Mediabestanden` `_Portály` `oreferrer`
Llama2 70B	32,000	No	32/639	`_Mediabestanden` `_Portály` `ederbörd`
Mistral 7B v0.3	32,000	No	53/637	`\uefc0` `});\r` `ও` `>?[<` `_febbra` `_uitgen`
Mixtral 8x7B	32,000	No	44/637	`\uefc0` `_/**\r` `ও` `];\r`
Rakuten 7B	48,000	No	66/957	`\uefc0` `_/**\r` `ও` `_febbra` `稲田大学`
Qwen1.5 32B	151,646	No	2450/2966	`_ForCanBeConvertedToF` `(stypy` `$PostalCodesNL`
Qwen1.5 72B Chat	151,646	No	2047/2968	`_ForCanBeConverted` `useRalative` `_typingsJapgolly`
StableLM2 12B	100,288	No	138/1997	`_ForCanBeConverted` `\tTokenNameIdentifier` `_StreamLazy`
Llama3 8B	128,256	No	556/2540	`_ForCanBeConverted` `ЎыцNЎыцN` `_CLIIIK` `krvldkf` `글상위`
Llama3 70B	128,256	No	462/2540	`$PostalCodesNL` `итися` `ılmaktadır` `ーション` `;\r\r\r\n`
Command R (35B)	255,029	Yes	306/5012	`AddLanguageSpecificText` `_ARStdSong` `目前尚未由人工引`
Command R+ (104B)	255,029	Yes	75/5012	`AddLanguageSpecificText` `tocguid` `ephritidae`
Gemma 2B	256,000	Yes	3161/5117	`हिंदीखरीदारी` `^(@)$_` `_coachTry` `_AcceptedLoading` `ICTOGRAM`
Gemma 7B	256,000	Yes	800/5117	`हिंदीखरीदारी` `EnglishChoose` `_queſto` `_stockfotografie` `▨`
Starcoder2 15B	49,152	No	128/968	`ittrLoremipumdolorsitametconsecteturadipiscingelitIntegervelvel`
Yi 9B	64,000	No	245/1278	`\\+::\\+` `mcited` `mabaochang` `nConsequently`
Jamba v0.1 (52B)	65,536	No	6/1280	`derrelsc` `]{}]{}` `ronicsystems`

Table 1: **Detection of under-trained tokens.** #Confirmed are the confirmed/tested numbers for the tokens tested in verification that are predicted with a maximal probability of $< 1\%$ across verification prompts. Examples were manually chosen for readability, similarity across models or for being particularly striking. Note that the leading '_' in tokens such as `_SolidGoldMagikarp` indicates a leading space.

*These models include a bias term in their final layer, which does not affect our results as we use their input embeddings.

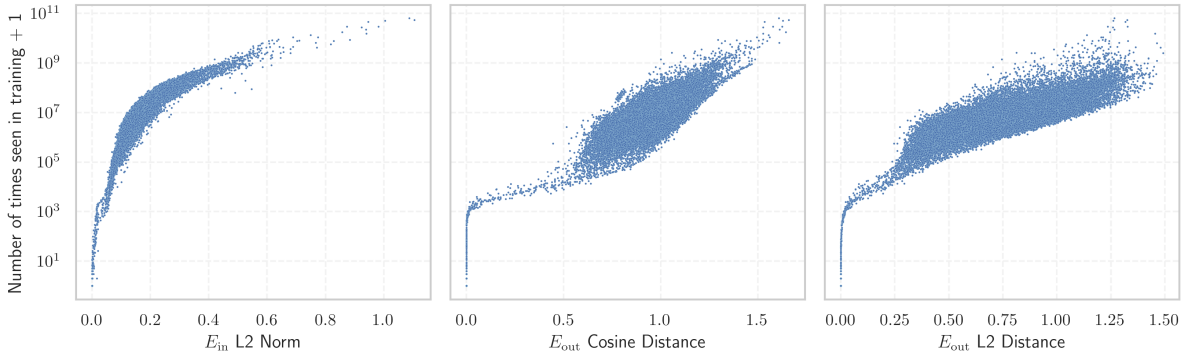

Figure 2: Under-trained token indicators are highly predictive of training data. The embedding-based under-trained token indicators for the OLMo v1.7 7B model and the number of times each token appears in the first epoch of training are shown. All indicators correlate strongly with the number of times a token is seen in training, not only at the expected lower values, but extending across ten orders of magnitude.

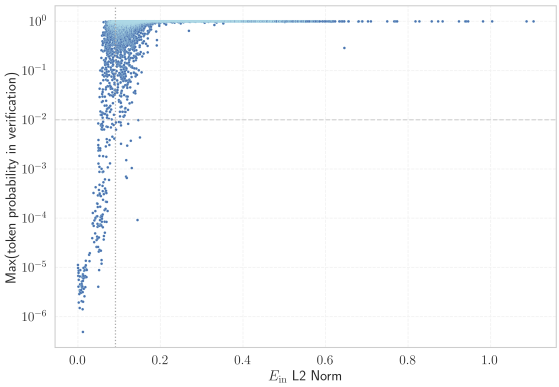

Figure 3: Under-trained token indicators are predictive of verification probability. The rate of successful verification ($p < 0.01$) correlates very highly with our proposed indicator, with no false positives at low values of the indicator scores and a low rate of false negatives. The dotted vertical line indicates the default 2% threshold used for verification.

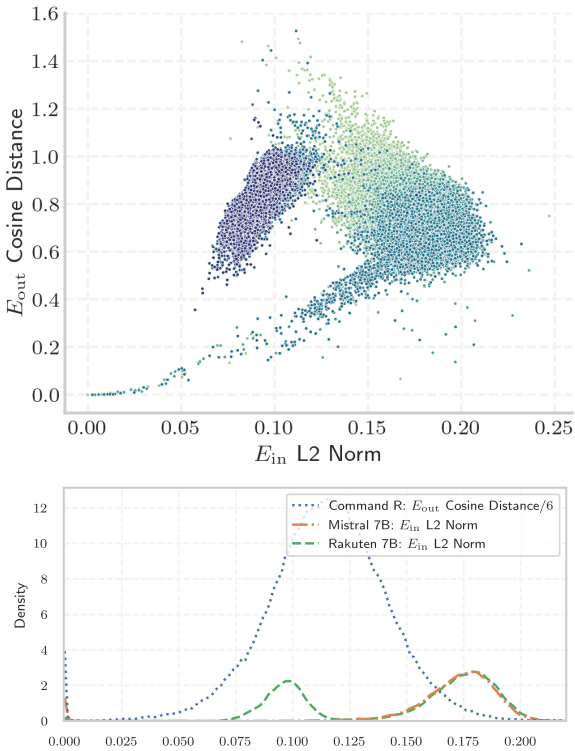

Figure 4: Comparison of indicators. The scatter plots are coloured by token ID, from light green to dark blue. *Top*: Rakuten 7B showing a separate cluster for added tokens, and high correlation near zero, showing that the two different indicators are similar in effectiveness. *Bottom*: In density plots, a clear peak appears near zero for most models, giving rise to a bimodal distribution.

configuration issues that can be identified using our methods, and are not exhaustive.

GPT-2 (Radford et al., 2019) introduced the framework for much of current-day LLM development, and the tokenizer has been re-used extensively. We confirm previous findings with a significant number of tokens related to (fragments of) usernames (e.g. `_TheNitrome`, `_RandomRedditor`). We also find a number of under-trained non-English tokens. Additionally, all ASCII control characters except for the newline character, but including the tab and carriage return characters, appear untrained. This suggests a potential mismatch in data normalization between training and inference.

GPT-J 6B (Wang and Komatsuzaki, 2021) and **Phi-2** (Microsoft, 2023) are independent models which both also use the GPT-2 tokenizer, and have significantly more under-trained tokens, likely due to their training data being further removed from the data used to train the tokenizer. These additional tokens include `_SolidGoldMagikarp`, which is not among verified candidates in GPT-2.

GPT-NeoX is an open-source library and associated family of models that uses a tokenizer with the same vocabulary size as GPT-2, but trained on the same 'The Pile' dataset also used for model training, and with added tokens for multiple spaces (Black et al., 2022). The GPT-NeoX 20B model has very few under-trained tokens, likely in part due to this alignment between tokenizer and model training, with the fragment `FFIRMED` showing up most consistently. The Pythia 6.7B model based on the same library (Biderman et al., 2023) also shows very similar results.

OLMo open language models (Groeneveld et al., 2024) also use the GPT-NeoX tokenizer, but have a much higher rate of under-trained tokens, including a wide range of punctuation-based tokens. We also detect over 200 unreachable tokens representing combinations of spaces and line breaks in the tokenizer, which appear to be caused by the aforementioned 'multiple spaces' tokens taking precedence. However, many of them appear to have been seen in training, based on both our indicators and training data statistics.[4]

Furthermore, we noticed that embedding-based indicators are not near zero for the GPT-NeoX and Pythia models, as well as v1 of the OLMo

[4]This was in part traced to a breaking change in `tokenizers` v0.14 (Luca Soldaini, personal communication).